OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

Name: R. Peeran Basha

Register Number: 192472244

Department / Class: CSE(AI)

Course: Object Oriented Programming with C++

Faculty: Jagadeesh K

Q1 – Call by Value, Reference and Pointer

Objective: Compare how a function receives an argument and whether changes are visible to the caller.

Concept and analysis:

- Call by value creates a separate parameter copy. Updating it changes only the local copy.
- Call by reference (`int&`) aliases the caller's variable. Updating it changes the original variable.
- Call by pointer (`int*`) receives an address. Dereferencing the pointer changes the object at that address.
- A `const Student&` is appropriate when a large object must be read without copying and must not be modified.

Design decision: For a single fee update, a function that returns the updated fee is easy to compose and test. For a sequence of updates, a reference-based in-place function avoids repeatedly assigning the returned value. This report uses a reference for the required update and shows a return-value alternative.

Predict-the-output program: `change` leaves `a` unchanged, increases `b` and `c` by 10; `bigger(a,c)=100` assigns through the returned reference to the larger variable; the three `show` calls use the default values as applicable.

C++17 Source Code

```
#include <iostream>
void change(int a, int& b, int* c) {
    a += 10;
    b += 10;
    *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) {
    std::cout << x + y + z << ' ';
}
int main() {
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\\n';
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\\n';
    show(1); show(1, 2); show(1, 2, 3);
    std::cout << '\\n';
}
```

Expected / Illustrative Output

```
1 12 13
100 13
16 13 6
```

Q1 – Completed concession program

Explanation: `applyByValue` receives a copy of the fee, so the caller's `hostelFee` remains 45000. `applyConcession` receives an alias to the caller's variable and subtracts 10%, producing 40500.

Complexity: Both approaches perform a constant number of arithmetic operations: O(1) time. A `long long` value copy uses O(1) extra space. A large object passed by value copies its fields and may cost O(size of object); reference and const-reference parameters pass an alias and avoid that object copy.

C++17 Source Code

```
#include <iostream>
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
long long applyAndReturn(long long fee, int percent) {
    return fee - fee * percent / 100;
}
int main() {
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value   : " << hostelFee << '\\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\\n';
    std::cout << "return-value alternative: "
              << applyAndReturn(45000, 10) << '\\n';
}
```

Expected / Illustrative Output

```
after call by value   : 45000
after call by reference: 40500
return-value alternative: 40500
```

Q2 – Return by Reference and Its Dangers

Objective: Return an array element by reference so the caller can update the original mark.

An `int&` return aliases the selected array element, so `highest(marks, n) += 2` changes the stored mark. A `const int&` allows reading but prevents assignment through that reference. Returning `int` returns a copy; changing the copy does not update the array.

Never return a reference to a local variable: the local object is destroyed when the function returns, leaving a dangling reference. Accessing it is undefined behaviour. Compilers commonly warn that a reference to a local variable is returned.

Vector condition: `push_back` or another growth operation may reallocate the vector's storage. References, pointers and iterators to its elements can then be invalidated. An index remains an integer value, but must still be checked against the vector's current size before use.

Alternatives: An index returns the position and is robust across reallocations if re-evaluated against the current vector. A pointer is direct but can dangle after reallocation. A reference is convenient but also can dangle after reallocation. Finding the maximum takes O(n) time and O(1) auxiliary space.

C++17 Source Code

```
#include <iostream>
int& highest(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
const int& highestReadOnly(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int highestCopy(const int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int main() {
    int marks[] = {78, 91, 84, 88};
    highest(marks, 4) += 2;
    std::cout << "Updated highest mark: " << highestCopy(marks, 4) << '\\n';
    std::cout << "Read-only highest mark: "
              << highestReadOnly(marks, 4) << '\\n';
}
```

Expected / Illustrative Output

```
Updated highest mark: 93
Read-only highest mark: 93
```

Q3 – Default Arguments and Function Overloading

Objective: Design courier charge functions with useful defaults and distinct overloads.

The original overload set is ambiguous because calls such as `charge(2)` can match multiple candidates after default arguments are supplied. A default argument does not create a separate overload; it fills omitted trailing parameters. Defaults must be supplied from right to left because trailing parameters are the ones that can be omitted.

Overload resolution prefers an exact match over a promotion, and a promotion over a standard conversion. `2` is an `int`; `2.0f` is a `float` and can convert to `double`. The explicit integer and double overloads below make these calls unambiguous. The Parcel overload handles a parcel object.

Assumption for this worked example: base rate is ₹40 per kg; express adds ₹20; insurance adds ₹10. Integer weight is interpreted in grams, double weight in kilograms. These rates are illustrative because the assignment sheet does not specify rates.

C++17 Source Code

```
#include <iostream>
struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};
double charge(double weightKg, bool express = false,
              bool insured = false) {
    return weightKg * 40.0 + (express ? 20.0 : 0.0)
                           + (insured ? 10.0 : 0.0);
}
double charge(int weightGrams) {
    return charge(weightGrams / 1000.0, false, false);
}
double charge(float weightKg) {
    return charge(static_cast<double>(weightKg), false, false);
}
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}
int main() {
    Parcel p{2.5, true, true};
    std::cout << "charge(2): " << charge(2) << '\\n';
    std::cout << "charge(2.0): " << charge(2.0) << '\\n';
    std::cout << "charge(2.0f): " << charge(2.0f) << '\\n';
    std::cout << "charge(1500): " << charge(1500) << '\\n';
    std::cout << "charge(2.0, true): " << charge(2.0, true) << '\\n';
    std::cout << "charge(parcel): " << charge(p) << '\\n';
}
```

Expected / Illustrative Output

```
charge(2): 0.08
charge(2.0): 80
charge(2.0f): 80
charge(1500): 0.06
charge(2.0, true): 100
```

```
charge(parcel): 130
```

Note: Integer inputs are interpreted as grams in this example. The displayed values are in the same illustrative currency units as the assumed rate.

Q4 – Specifying a Class with an Array Inside It

Objective: Build a SavingsAccount class that encapsulates account data and keeps only the most recent 10 transactions.

The account number, balance, transaction array and count are private so callers cannot bypass validation or corrupt the transaction history. Public methods provide controlled deposits, withdrawals and statement display.

Fixed-array strategy used here: when full, shift entries one position left and write the new transaction at the end. This costs O(k) per insertion for k stored entries (at most 10 here). A circular buffer instead overwrites the oldest slot using a moving start index, so insertion is O(1), but display must traverse in logical order.

Design decision for 50 transactions: use a circular buffer because it avoids shifting 49 entries for every new transaction. The storage remains fixed at 50 records and insertion remains O(1). A `std::deque` is another suitable option with push_back/pop_front.

C++17 Source Code

```
#include <iostream>
#include <string>
class SavingsAccount {
    int accountNumber;
    double balance;
    std::string transactions[10];
    int count = 0;
public:
    SavingsAccount(int number, double opening)
        : accountNumber(number), balance(opening) {}
    void deposit(double amount) {
        if (amount <= 0) return;
        balance += amount;
        record("Deposit: +" + std::to_string(amount));
    }
    void withdraw(double amount) {
        if (amount <= 0 || amount > balance) return;
        balance -= amount;
        record("Withdrawal: -" + std::to_string(amount));
    }
    void miniStatement() const;
private:
    void record(const std::string& entry) {
        if (count == 10) {
            for (int i = 1; i < 10; ++i)
                transactions[i - 1] = transactions[i];
            --count;
        }
        transactions[count++] = entry;
    }
};
void SavingsAccount::miniStatement() const {
    std::cout << "Account: " << accountNumber
              << " | Balance: " << balance << '\\n';
    std::cout << "Recent transactions:\\n";
```

```
    for (int i = 0; i < count; ++i)
        std::cout << transactions[i] << '\\n';
}
int main() {
    SavingsAccount account(1001, 5000);
    account.deposit(500);
    account.withdraw(250);
    account.deposit(1000);
    account.miniStatement();
}
```

Expected / Illustrative Output

```
Account: 1001 | Balance: 6250
Recent transactions:
Deposit: +500.000000
Withdrawal: -250.000000
Deposit: +1000.000000
```

Q5 – Static Members and Arrays of Objects

Objective: Use class-wide state to allocate bus seats and count issued tickets.

`issued` and `CAPACITY` are static because they belong to the class rather than to an individual ticket. The static data member is defined once outside the class. A static member function has no `this` pointer, so it cannot directly access a particular object's non-static `seat` or `name`.

An array of objects constructs each element. The constructor runs once per element, so the shared count persists and increases across loop iterations and function calls. If a function is called twice, the count does not reset unless the program explicitly resets it; the next call continues from the previous count.

With a global counter, unrelated code could modify the value and weaken encapsulation. A separate TicketOffice object is more flexible when two buses need independent capacities and counts. Static members are appropriate when exactly one shared counter is required.

C++17 Source Code

```
#include <iostream>
#include <string>
class BusTicket {
    static int issued;
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) seat = ++issued;
    }
    static int count() { return issued; }
    void print() const {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat)
                           : std::string("Waiting")) << '\\n';
    }
};
int BusTicket::issued = 0;
int main() {
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected / Illustrative Output

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

Q5 – Predict the Output (Token)

Prediction: the three array objects receive IDs 101, 102 and 103. The extra object receives 104. `issued()` returns `next - 100`, which is 4.

Constructor calls: four in total—three for `counter[3]` and one for `extra`. The static counter begins at 100 because its out-of-class definition initializes it to 100 before `main` creates objects.

C++17 Source Code

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' '
              << counter[2].get() << '\\n'
              << extra.get() << ' '
              << Token::issued() << '\\n';
}
```

Expected / Illustrative Output

```
101 103
104 4
```

Q6 – Friend Functions and Inline Functions

Objective: Compare two Time objects without exposing their private representation.

A friend function is not a member function, but the class grants it access to private members. It is called like an ordinary function: `minutesBetween(a, b)`. A member function would be called through an object, such as `a.minutesTo(b)`. Friend functions are useful when an operation treats both operands symmetrically.

Making `h` and `m` public would remove encapsulation: callers could create invalid values such as 27 hours or 90 minutes. A constructor and controlled interface can preserve the class invariant.

`inline` permits a function definition to appear in multiple translation units under the One Definition Rule and is also a request that the compiler may substitute the function body at a call site. It does not guarantee substitution. Modern compilers decide based on optimisation and context. Tiny accessors may be declared inline, but performance must be measured rather than assumed.

Timing evidence: no execution measurements are available in the supplied assignment document or in this report environment. The benchmark code below can be compiled with `-O0` and `-O2`; record actual wall-clock results on the student's machine. Do not report invented timings.

C++17 Source Code

```
#include <iostream>
#include <cstdlib>
class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b) {
    int first = a.h * 60 + a.m;
    int second = b.h * 60 + b.m;
    return std::abs(first - second);
}
bool earlier(const Time& a, const Time& b) {
    return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
    Time departure(9, 45), arrival(13, 10);
    std::cout << minutesBetween(departure, arrival) << ' '
              << std::boolalpha
              << earlier(arrival, departure) << '\\n';
    Time noonBefore(11, 59), noonAfter(12, 1);
    std::cout << minutesBetween(noonBefore, noonAfter) << ' '
              << earlier(noonBefore, noonAfter) << '\\n';
    std::cout << minutesBetween(noonBefore, noonBefore) << ' '
              << earlier(noonBefore, noonBefore) << '\\n';
}
```

Expected / Illustrative Output

```
205 false
2 true
0 false
```

Q6 – Inline benchmark template

Use the following separate benchmark to compare an inline and non-inline operation. Run each build under the same machine conditions. The compiler may optimise away unused work, so the accumulated result is printed to keep the calculation observable.

Compile: `g++ -std=c++17 -O0 inline_benchmark.cpp -o bench_O0` and `g++ -std=c++17 -O2 inline_benchmark.cpp -o bench_O2`. Run both executables and record the measured seconds. The actual values depend on processor, compiler, operating system and optimisation.

C++17 Source Code

```
#include <iostream>
#include <chrono>
using clock_type = std::chrono::steady_clock;
inline int inlineAdd(int a, int b) { return a + b; }
int normalAdd(int a, int b) { return a + b; }
int main() {
    constexpr int N = 10000000;
    volatile long long total = 0;
    auto start = clock_type::now();
    for (int i = 0; i < N; ++i) total += inlineAdd(i, 1);
    auto middle = clock_type::now();
    for (int i = 0; i < N; ++i) total += normalAdd(i, 1);
    auto finish = clock_type::now();
    std::cout << "inline seconds: "
      << std::chrono::duration<double>(middle-start).count() << '\\n';
    std::cout << "normal seconds: "
      << std::chrono::duration<double>(finish-middle).count() << '\\n';
    std::cout << "check: " << total << '\\n';
}
```

Expected / Illustrative Output

```
Output format (timings vary by computer):
inline seconds: <measured value>
normal seconds: <measured value>
check: 100000010000000
```

Q7 – Diagnose, Redesign and Validate

Selected program: Q4 SavingsAccount transaction history using a fixed array with shifting when the array is full.

A. Diagnose the limitation: once the array contains 10 transactions, every new transaction shifts the remaining 9 entries left before inserting the new one. The original design therefore performs O(k) assignments per new transaction, where k is the number of stored entries. With a larger history such as 50, this repeated shifting is unnecessary work.

B. Measurable objective: retain the most recent 50 transactions while making insertion O(1) and keeping storage bounded at 50 entries.

C. Revised design: use a circular buffer with a fixed array, a `start` index pointing to the oldest entry, and a `count`. When there is room, append at `(start + count) % CAPACITY`. When full, overwrite `start` and advance `start = (start + 1) % CAPACITY`. Display from oldest to newest using the same modulo calculation.

D. Prediction: the original full-buffer insertion performs up to 49 element moves for a 50-entry history; the circular-buffer insertion performs a fixed number of index and assignment operations (O(1)). Memory remains O(50), i.e. O(k) for configured capacity k.

E. Implementation and validation: the following implementation demonstrates the redesigned behaviour. The output confirms that after 52 transactions, only transactions 3 through 52 remain. This is a deterministic functional validation; timing measurements should be collected on the student's own compiler and computer.

F. Final evaluation: the circular buffer meets the objective by eliminating the shifting work and preserving bounded storage. Its trade-off is more complex indexing and display logic. A simple shifting array may remain preferable for a very small history where clarity matters more than insertion cost.

C++17 Source Code

```
#include <iostream>
#include <string>
class TransactionHistory {
    static const int CAPACITY = 50;
    std::string entries[CAPACITY];
    int start = 0;   // index of oldest entry
    int count = 0;
public:
    void add(const std::string& value) {
        if (count < CAPACITY) {
            int pos = (start + count) % CAPACITY;
            entries[pos] = value;
            ++count;
        } else {
            entries[start] = value;       // overwrite oldest
            start = (start + 1) % CAPACITY;
        }
    }
    void display() const {
        for (int i = 0; i < count; ++i) {
```

```
            int pos = (start + i) % CAPACITY;
            std::cout << entries[pos] << '\\n';
        }
    }
    int size() const { return count; }
};
int main() {
    TransactionHistory history;
    for (int i = 1; i <= 52; ++i)
        history.add("Transaction " + std::to_string(i));
    std::cout << "Stored: " << history.size() << '\\n';
    history.display();
}
```

Expected / Illustrative Output

```
Stored: 50
Transaction 3
Transaction 4
...
Transaction 52
(The full program prints all 50 retained transactions.)
```